

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Week #12: Namespaces





Namespaces - 01

- A namespace is a mechanism for expressing logical grouping.
- Some declarations when logically belong together, put them in a common namespace
- Example.
- Function `expr()` must be declared and later defined.

```
namespace Parser
{
    double expr(bool);
    double prim(bool get);
    double term(bool get);
}
```

Namespaces - 02



- Defining members
- A member can be declared within a namespace definition
- The member can be defined later using scope resolution operator
- You cannot define a new member to the name space without including it into the namespace first

```
double Parser::expr(bool)
{
    /* Develop the code */
}
```

Namespaces - 03



- A namespace is a scope
- The usual scope rules hold for namespace
- If a name is previously declared in the namespace it can be used without any error

```
double Parser::term(bool get)
{
    double left = prim(get);
    for( ; ; )
    {
        switch( Lexer::curr_tok )
        {
            case Lexer::MUL:
                // do something
                break;
        }
    }
}
```

Namespaces - 04



- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
double Parser::term(bool get)
{
    if(get) Lexer::get_token();
    switch( Lexer::curr_tok )
    {
        case Lexer::NUMBER:
            Lexer::get_token();
            return Lexer::number_value;
        case Lexer::NAME:
            if( Lexer::get_token() == Lexer::ASSIGN )
                V = expr(true);
            return v;
    }
}
```

Namespaces - 05



- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
int main(int argc, char *argv)
{
    int a = 0, j = 0, k = 0;
    std::cin >> a;
    std::cin >> j;
    std::cin >> k;
    std::cout << a;
    std::cout << j;
    std::cout << k;
}
```

Namespaces - 06



- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
using namespace std;

int main(int argc, char *argv)
{
    int a = 0, j = 0, k = 0;
    cin >> a;
    cin >> j;
    cin >> k;
    cout << a;
    cout << j;
    cout << k;
}
```


Namespaces - 07



- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
double Parser::term(bool get)
{
    using Lexer::get_token;
    using Lexer::curr_tok;
    using Lexer::NUMBER;
    using Lexer::number_value;
    if(get) get_token();
    switch( curr_tok )
    {
        case NUMBER:
            get_token();
            return number_value;
        case NAME:
            if( get_token() == Lexer::ASSIGN )
                V = expr(true);
            return v;
    }
}
```



Namespaces - 08

- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
namespace Parser
{
    double prim(bool);
    double term(bool);
    double expr(bool);

    using Lexer::get_token;
    using Lexer::curr_tok;
    using Lexer::NUMBER;
    using Lexer::number_value;
}
```

Namespaces - 09



- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
double Parser::term(bool get)
{
    if(get) get_token();
    switch( curr_tok )
    {
        case NUMBER:
            get_token();
            return number_value;
        case NAME:
            if( get_token() == Lexer::ASSIGN )
                v = expr(true);
            return v;
    }
}
```



Namespaces - 10

- When a name is frequently used outside its namespace, it can be tedious task to repeat the qualifying namespace name.

```
namespace Parser
{
    double prim(bool);
    double term(bool);
    double expr(bool);

    using namespace Lexer;
    using namespace error;
}
```

Namespaces - 11



- Multiple interfaces

```
class Document: public Interface1::Printable,
               public Interface2::Saveable
{
    private:
        string content;

    public:
        Document(const string& text) : content(text) {}

        void print() const override
        {
            cout << "Printing Document: " << content <<
endl;
        }

        void save() const override
        {
            cout << "Saving Document: " << content << endl;
        }
};
```

```
namespace Interface1
{
    class Printable
    {
        public:
            virtual void print() const = 0;
            virtual ~Printable() = default;
    };
}

namespace Interface2
{
    class Saveable
    {
        public:
            virtual void save() const = 0;
            virtual ~Saveable() = default;
    };
}
```

Namespaces - 12



- **Multiple interfaces**

```
int main(int argc, char *argv[])
{
    Document doc("Hello, world!");

    Interface1::Printable* printableDoc = &doc;
    printableDoc->print();

    Interface2::Saveable* saveableDoc = &doc;
    saveableDoc->save();

    return 0;
}
```

```
$ g++ interface.cc
$ ./a.out
Printing Document: Hello, world!
Saving Document: Hello, world!
```

Namespaces - 13



- **Namespace 1**
- **Contains the Printable interface, which defines a pure virtual function print().**
- **Any class inheriting from Printable must implement the print() function.**
- **Namespace 2**
- **Contains the Saveable interface, which defines a pure virtual function save().**
- **Any class inheriting from Saveable must implement the save() function.**

Namespaces - 14



- **Class Document:**
- **Implements both the Printable and Saveable interfaces.**
- **It provides its own implementation for the print() and save() functions.**

Namespaces - 15



- Avoiding name clashes
- When we use only single global scope it is unnecessarily difficult to compose a program out of separate parts.
- The problem is that the supposedly separate parts, each define the same names.
- When combined into same program, these names clash.

Namespaces - 16



- Consider the example
- Given these, a third party cannot use both my.h and your.h

```
// my.h
char f(char);
int f(int);
class String
{
    // Implement something
};
```

```
// your.h
char f(char);
int f(int);
class String
{
    // Implement something
};
```

Namespaces - 17



- Solution is

```
namespace My
{
    char f(char);
    int f(int);
    class String
    {
        // Implement something
    };
}
```

```
namespace Your
{
    char f(char);
    int f(int);
    class String
    {
        // Implement something
    };
}
```



Un-named Namespaces - 18

- It is often useful to wrap a set of declarations in a namespace simply to protect against the possibility of name clashes
- Aim to preserve locality of code than to present an interface to users

```
namespace Mine
{
    int a;
    void f();
    int g();
}
```

```
namespace $$$
{
    int a;
    void f();
    int g();
}
using namespace $$$;
```



Namespaces Aliases- 19

- Provide short aliases to long namespace names

```
namespace American_Telephone_And_Telegraph
{
    int a;
    void f();
    int g();
}

namespace ATT =American_Telephone_And_Telegraph;

ATT::a = 10;
```

Namespaces composition - 20



- We want to compose an interface out of existing interfaces

```
namespace His_data
{
    int a;
    void f();
    int g();
}

namespace her_data
{
    // declare some variables
}

namespace my_lib
{
    using namespace his_data;
    using namespace her_data;
    void my_function();
}
```

Namespaces & Overloading - 21



- Overloading

```
namespace A
{
    void f(int);
}

namespace B
{
    void f(char);
}

// user.c
using namespace A;
using namespace B;

void g()
{
    f('g'); // calls B::f(char)
    f(10);  // calls A::f(int)
}
```

Namespaces are open - 22



- Following example

```
namespace A
{
    int f(void);
}
```

```
namespace A
{
    void g(char);
}
```

A has two members `int f(void)` and `void g(char)`



Questions?





Thank You

